

ML LAB EXPT – 6

Aim of the Experiment: Develop a program to demonstrate the working of Linear Regression and Polynomial Regression. Use Boston Housing Dataset (BostonHousing.csv) for Linear Regression and Auto MPG Dataset (auto-mpg.csv) (for vehicle fuel efficiency prediction) for Polynomial Regression.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, r2_score

# Set up the plots
plt.figure(figsize=(15, 6))

## Part 1: Linear Regression on Boston Housing Dataset
print("\n=== Linear Regression on Boston Housing Dataset ===")

# Load Boston Housing data
boston_data = pd.read_csv('BostonHousing.csv')
X_boston = boston_data[['rm']] # Using average number of rooms as feature
y_boston = boston_data['medv'] # Median home value as target

# Split data
X_train_b, X_test_b, y_train_b, y_test_b = train_test_split(X_boston,
y_boston, test_size=0.2, random_state=42)

# Train linear regression model
lin_reg = LinearRegression()
lin_reg.fit(X_train_b, y_train_b)

# Make predictions
y_pred_b = lin_reg.predict(X_test_b)

# Evaluate
mse_b = mean_squared_error(y_test_b, y_pred_b)
r2_b = r2_score(y_test_b, y_pred_b)

print(f"Linear Regression Results:")
print(f"Mean Squared Error: {mse_b:.2f}")
```

```

print(f"R-squared: {r2_b:.2f}")

# Plot results
plt.subplot(1, 2, 1)
plt.scatter(X_test_b, y_test_b, color='blue', label='Actual')
plt.plot(X_test_b, y_pred_b, color='red', linewidth=2, label='Predicted')
plt.xlabel('Average Number of Rooms (RM)')
plt.ylabel('Median Home Value ($1000s)')
plt.title('Boston Housing: Linear Regression')
plt.legend()

## Part 2: Polynomial Regression on Auto MPG Dataset
print("\n=== Polynomial Regression on Auto MPG Dataset ===")

# Load Auto MPG data
auto_data = pd.read_csv('auto-mpg.csv')

# Clean data - handle missing values and convert horsepower to numeric
auto_data['horsepower'] = pd.to_numeric(auto_data['horsepower'],
errors='coerce')
auto_data.dropna(inplace=True)

# Select features and target
X_auto = auto_data[['weight']] # Using vehicle weight as feature
y_auto = auto_data['mpg']      # Miles per gallon as target

# Split data
X_train_a, X_test_a, y_train_a, y_test_a = train_test_split(X_auto,
y_auto, test_size=0.2, random_state=42)

# Create polynomial regression model (degree=2)
poly_reg = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)
poly_reg.fit(X_train_a, y_train_a)

# Make predictions
y_pred_a = poly_reg.predict(X_test_a)

# Evaluate
mse_a = mean_squared_error(y_test_a, y_pred_a)
r2_a = r2_score(y_test_a, y_pred_a)

print(f"\nPolynomial Regression (degree=2) Results:")

```

```

print(f"Mean Squared Error: {mse_a:.2f}")
print(f"R-squared: {r2_a:.2f}")

# Plot results
plt.subplot(1, 2, 2)
plt.scatter(X_test_a, y_test_a, color='blue', label='Actual')

# Sort values for smooth curve
X_sorted = np.sort(X_test_a.values, axis=0)
y_sorted = poly_reg.predict(X_sorted)
plt.plot(X_sorted, y_sorted, color='red', linewidth=2, label='Polynomial
Fit')

plt.xlabel('Vehicle Weight')
plt.ylabel('Miles Per Gallon (MPG)')
plt.title('Auto MPG: Polynomial Regression (degree=2)')
plt.legend()

plt.tight_layout()
plt.show()

```

Output:

```

=== Linear Regression on Boston Housing Dataset ===
Linear Regression Results:
Mean Squared Error: 46.14
R-squared: 0.37

```

```

=== Polynomial Regression on Auto MPG Dataset ===

Polynomial Regression (degree=2) Results:
Mean Squared Error: 16.69
R-squared: 0.67

```

